Tactile Design System: Architecture, Current UI Surfa

Prepared for the Reaktor, BestBuds, and Reaktor Desktop codebase.

Date: 2026-04-14

Executive Summary

The Tactile Design System should not be built as a static JSON server-driven UI system. The central architectural decision is that React becomes the programming model for dynamic UI composition, while the Tactile host tree becomes an internal reconciler output. Product engineers and AI agents should write React components, not JSON layout documents and not a partial expression language.

The target model is:

```
React components
  -> custom React reconciler
  -> TactileHostTree
  -> Reaktor FFI bridge
  -> StateFlow<TactileHostNode>
  -> TactileComposeRenderer
  -> Compose Multiplatform
```

For React-native terminal targets, the path is simpler:

```
React components
  -> @reaktor/tactile-rsd
  -> React Strict DOM
  -> DOM / React Native
```

The Tactile host tree is an implementation detail. It is similar in role to a shadow tree or host tree in other renderer systems. It is not a public schema language. Its purpose is to make the React composition model renderable through the native Compose Multiplatform design system.

Reaktor Graph remains the app orchestration layer. It should continue to own navigation, route payloads, graph ports, dependency scopes, services, repositories, and app lifecycle. Tactile should plug into graph rendering as a sibling to the current ComposeContent path, not replace the graph runtime.

BestBuds mobile should be the first product transition target. The transition should begin with leaf components and product design tokens, not whole screens. Reaktor Desktop should be upgraded later as tooling for host-tree inspection, component catalogues, skin switching, and route preview.

Why Not Static SDUI

Static SDUI systems often start with a clean idea: the server sends a tree of UI components, and the client renders those components. This is safe, inspectable, and easy to cache at the beginning.

The problem is that real product UI quickly needs features that are not simply a tree:

- conditionals
- loops
- local state
- form binding
- validation
- optimistic updates
- loading states
- effects and lifecycle
- component composition
- feature flags and experiments
- navigation actions
- cross-component coordination

When those needs appear, a static SDUI system tends to grow a partial programming language. JSON receives fields like ‘visibleWhen‘, ‘forEach‘, ‘bind‘, ‘onChange‘, ‘validate‘, ‘computed‘, and ‘action‘. The design then becomes a custom language without the tooling, debuggability, libraries, and mental model of a real programming language.

The second failure mode is separation. Logic becomes split between the SDUI document, server-side builders, and client widgets. It becomes unclear whether behavior belongs in the schema or in the widget. Different widgets start interpreting similar configuration differently. The server begins to depend on client implementation quirks, and clients begin to encode product-specific behavior in generic widgets.

The Tactile approach should avoid this. React is already a complete component language. It has functions, conditionals, loops, composition, local state, effects, context, and mature tooling. The reconciler can make React target Reaktor’s native Compose runtime without inventing another programming language.

This does not mean JSON or FlexBuffers disappear. They may still exist as internal encodings for the reconciler commit protocol. The difference is that the authoring model is not JSON. The authoring model is React.

Target Architecture

The system has four major runtime surfaces:

1. React composition surface.
2. Tactile host tree and reconciler bridge.

3. Native Compose Multiplatform renderer.
4. React Strict DOM renderer.

The responsibilities should remain strict.

React owns:

- component composition
- conditionals
- loops
- local UI state
- hooks
- effects
- component abstraction
- handler definitions
- AI-generated component source

Tactile owns:

- host component contracts
- design tokens
- skins and visual languages
- interaction state
- physics and springs
- haptics and sound hooks
- shader field integration
- adaptive layout primitives
- renderer-neutral semantics

Reaktor Graph owns:

- route graph
- back stack
- route payloads
- graph ports
- dependency scopes
- repositories
- services
- app lifecycle
- app-level action registry

Compose owns:

- native layout
- native rendering
- accessibility semantics
- pointer, hover, focus, and drag input
- native haptics
- native spring and animation execution

- Skia shader execution

React Strict DOM owns:

- DOM and React Native terminal rendering
- ‘html.*‘ primitives
- ‘css.create‘ style definitions
- ARIA and platform semantic mapping for the React path

The most important performance rule is that React must not drive tactile animation frames. A press, drag, hover, shader ripple, haptic impulse, or spring simulation should not require a per-frame JS to Kotlin round trip. React should describe structure and semantic state. Kotlin and Compose should run the tactile runtime locally.

Core Runtime

The core runtime lives in Kotlin and should be renderer-independent wherever possible. The same concepts may have TypeScript mirrors for the React Strict DOM path, but the native Compose path should run the tactile runtime locally.

Interaction Snapshot

The original simple design used a sealed state such as ‘Created‘, ‘Focused‘, ‘Hovered‘, and ‘Pressed‘. That model is too narrow. A real control can be focused and hovered and pressed at the same time. A drag can begin while a press is active. Disabled state is orthogonal to hover and focus.

Use a snapshot with concurrent flags:

```
data class InteractionSnapshot(
    val focused: Boolean = false,
    val hovered: Boolean = false,
    val pressed: Boolean = false,
    val dragging: Boolean = false,
    val enabled: Boolean = true,
    val pointerPosition: Offset? = null,
    val pressOrigin: Offset? = null,
    val velocity: Velocity? = null,
    val pressDurationMs: Long = 0,
    val pressCount: Int = 0
)
```

The snapshot is the local state the renderer and skin consume. It is not a React state object and should not be updated across the FFI bridge every frame.

Interaction Controller

‘InteractionController‘ should be a native Kotlin controller created by the Compose renderer for interactive host nodes.

It should receive events such as:

- pointer down
- pointer up
- pointer move
- hover enter
- hover exit
- focus gain
- focus loss
- drag start
- drag move
- drag stop
- enabled state change

It should expose a ‘StateFlow<InteractionSnapshot>‘. Visuals, haptics, shader impulses, and local tactile reactions read the snapshot.

Physical Engine

The physical engine should start with renderer-neutral math:

```
data class MaterialPhysics(
    val mass: Float,
    val stiffness: Float,
    val damping: Float,
    val friction: Float
)
```

The engine should define presets such as Feather, Rubber, Wood, Steel, and Stone. These presets are not just animation constants. They express how a material should feel across press depth, rebound, drag, impact, and haptic response.

The engine should expose pure spring simulation, and Compose adapters should map material physics into Compose spring specs where appropriate. This allows native animation APIs to remain responsible for frames.

Tactile Field

The Tactile field is the decoupling layer for cross-element reactions.

When a button is pressed, the button should not know which background shader is active. It should emit an impulse into a shared field:

```
data class Impulse(
    val origin: Offset,
    val force: Float,
    val material: MaterialPhysics,
    val timestamp: Long
```

```
)
```

Other elements and shaders can read the field. This supports effects such as a background responding to a card press or a button press causing nearby elements to shift without direct component-to-component coupling.

This is especially important for the design goal of a living UI system. The field lets the UI feel connected without making every component know about every other component.

Sensory Runtime

Haptics and sound should be abstracted behind adapters:

```
interface HapticEngine {
    fun play(pattern: HapticPattern)
}
```

Haptic patterns should support fixed patterns such as light click and success, and dynamic patterns based on mass and velocity. The Compose renderer calls haptics from native interaction events. React handlers should not trigger low-level haptics directly for ordinary tactile interactions.

Shader Runtime

Skia shader support should be part of the Compose path and optional for web/canvas targets based on capability. The shader layer should read the Tactile field, not individual components.

The shader runtime should support:

- shader capability detection
- shader uniform derivation from ‘TactileField‘
- fallback rendering when shaders are unavailable
- no direct coupling between a component and a shader implementation

Skins And Tokens

BestBuds currently uses three styling systems at once:

- ‘BestBudsTheme‘, backed by old ‘reaktor-ui‘ ‘Theme‘.
- ‘BestBudsTokens‘, with ‘Spacing‘, ‘Corners‘, ‘BBColors‘, ‘AppGradient‘, and ‘ChatGradient‘.
- Direct Material3 values inside screens and product components.

The Tactile system should unify these through a product skin.

The skin interface should be renderer-neutral:

```
interface DesignLanguage {
    val name: String
    val defaultMaterial: MaterialPhysics
    fun tokens(base: DesignTokens): TactileTokens
    fun buttonVisuals(variant: String, size: String, snapshot: InteractionSnapshot): ButtonVisuals
    fun cardVisuals(variant: String, snapshot: InteractionSnapshot): CardVisuals
    fun surfaceVisuals(elevation: Float, snapshot: InteractionSnapshot): SurfaceVisuals
    fun inputVisuals(state: String, snapshot: InteractionSnapshot): InputVisuals
    fun textVisuals(style: String): TextVisuals
}
```

Visual specs must not contain Compose ‘Color‘, CSS strings, or platform-specific types. They should use raw values such as ARGB integers, floats, and semantic scalar values. Compose and RSD renderers convert those specs into platform-specific APIs.

‘BestBudsSkin‘ should be derived from:

- ‘BestBudsTheme‘
- ‘BBColors‘
- ‘Spacing‘
- ‘Corners‘
- ‘AppGradient‘
- ‘ChatGradient‘
- the observed card, chip, search, event, and chat visual patterns

This creates a bridge from today’s Compose UI to the future Tactile runtime without forcing a whole-screen rewrite first.

Host Tree And Reconciler

The host tree is the internal representation produced by the React reconciler.

It should not be public authoring syntax. Product engineers write React components. AI agents generate React components. The reconciler produces host nodes.

The Kotlin shape can start as:

```
data class TactileHostNode(
    val id: Int,
    val type: String,
    val props: Map<String, TactileValue>,
    val handlers: Map<String, HandlerRef>,
    val children: List<TactileHostNode>
)
```

The ‘type‘ field should use stable, versioned host component IDs:

```
reaktor.tactile.Button@1
```

```
reaktor.tactile.Text@1
reaktor.tactile.Card@1
reaktor.tactile.ListDetail@1
```

Versioning matters. If the Button host contract changes incompatibly, introduce ‘reaktor.tactile.Button@2‘. Do not silently break old bundles.

Handlers must be references, not serialized lambdas:

```
data class HandlerRef(val id: String)
```

On the JS side, a handler registry maps IDs to functions. On the Kotlin side, Compose dispatches native events back to Hermes using the handler ID. This keeps the host tree data-only while still letting React own handler logic.

The event round trip is:

```
User presses button
  -> Compose pointer input
  -> InteractionController updates snapshot
  -> TactileField receives impulse
  -> native haptics and spring animation run locally
  -> on release, Compose calls bridge.dispatchEvent(handlerId, payload)
  -> Hermes invokes registered JS handler
  -> React setState or action call runs
  -> custom reconciler commits host tree
  -> Kotlin StateFlow<TactileHostNode> updates
  -> Compose recomposes
```

For MVP, a full host tree commit using JSON is acceptable because it is simple and debuggable. After correctness is established, move to FlexBuffer full-tree commits. Only after that should op batches be introduced.

The sequence should be:

1. JSON full-tree commit.
2. FlexBuffer full-tree commit.
3. FlexBuffer op batches: create, insert, update, remove, reorder.

Adaptive Layout Architecture

The current graph renderer renders the top route in a graph. This is enough for compact screens. It is not enough for multi-pane layouts.

BestBuds chat is the key case:

- compact mode: ‘/chats‘ pushes ‘/chats/{id}‘;
- expanded mode: ‘/chats‘ and ‘/chats/{id}‘ should be visible at the same time;
- extra pane mode: friend profile or group profile can appear as a third pane.

The missing primitive is:

```
@Composable
fun GraphRouteContent(
    graph: Graph,
    route: RouteNode<*, *>,
    isFocused: Boolean = true
)
```

‘GraphContent(graph)‘ renders the current top route. ‘GraphRouteContent(graph, route)‘ renders a specific route. Adaptive pane containers need the second primitive.

The rule should be:

- Reaktor Graph owns route state and navigation.
- Compose Adaptive owns pane presentation.
- Tactile owns the renderer-neutral adaptive host component contract.

For a list-detail layout, the Tactile host component can represent:

- list pane
- detail pane
- extra pane
- collapse policy
- focus policy

Compose maps this to Material3 adaptive pane scaffolds. RSD maps it to responsive DOM/native layout. The route graph does not need to adopt Compose Navigation.

BestBuds Mobile UI Surface

Graph And Navigation

The BestBuds root graph is defined in:

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/BestBuds.kt‘

The root graph attaches data nodes:

- ‘MessageRepository‘
- ‘UserRepository‘
- ‘ChatInteractor‘
- ‘SocialRepository‘
- ‘UserInteractor‘

It then creates child graphs:

- Chat

- Profile
- Campaign
- Discover
- Events
- Friends
- Dev

The ‘/home‘ container uses ‘BottomNavigationContainer‘ with bottom nav keys:

- Chat
- Campaign
- Discover
- Events

Friends, Profile, and Dev are present as child graphs, but they are accessed through the top bar menu and overflow behavior. The top bar is implemented by ‘BestBudsTopBar‘.

The Events graph contains a nested ‘TabbedContainer‘ for Private and Public event graphs.

Start Screen

File:

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/StartScreen.kt‘

The start screen contains:

- typewriter translation branding
- Apple login button
- Google login button
- impersonated login avatar row
- auto-navigation for active users
- status-based user navigation

Important current behavior:

- ‘UserStatus.ONBOARDING‘ currently navigates to the chat list edge with a TODO comment saying to switch to the onboarding edge after testing.

Migration implications:

- This screen touches auth and app entry flow. It should not be the first React/Tactile route.
- The typewriter component is a good candidate for a Tactile animation primitive later.
- The impersonation avatars are good image/avatar leaf candidates.

Onboarding

Files:

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/onboarding/OnboardingScreen.kt‘

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/onboarding/OnboardingViews.kt‘

The onboarding UI contains:

- progress bar
- question count
- skip button
- centered question text
- true/false answer controls
- single-select grid
- integer-range slider
- back and submit/finish buttons
- service-backed response loading and submission

Migration implications:

- This is an excellent proof for React state and form logic, but only after the host component set and event bridge are stable.
- It exercises state, lists, conditional rendering, local response state, service calls, and navigation.
- It should migrate after simpler card/list screens.

Chat List

File:

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/chats/ChatsScreen.kt‘

The chat list UI contains:

- user gate through ‘UserRepository‘
- data fetch through ‘ChatInteractor‘
- ‘LazyColumn‘
- ‘ChatRow‘ for each chat
- route dispatch to ‘/chats/{id}‘ with ‘ChatPayload‘

Migration implications:

- ‘ChatRow‘ should migrate before ‘ChatListScreen‘.
- Route dispatch must remain graph-owned.
- The React component should call a named action or typed route action; it should not mutate graph internals directly.

Chat Detail

Files:

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/chats/ChatScreen.kt‘

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/chats/ChatViews.kt‘

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/chats/views/MessageView.kt‘

The chat detail UI contains:

- route payload reading
- open chat lifecycle effect
- close chat cleanup
- chat state render
- message list state
- older message pagination
- reverse ‘LazyColumn‘
- date headers
- top app bar with avatar/title/subtitle/back/call
- title click navigation to friend or group profile
- message input
- sender and receiver message bubbles
- group sender labels
- horizontal draggable message bubbles

Migration implications:

- Chat is the highest-value Tactile target because it contains tactile gestures, input, list behavior, navigation, and adaptive potential.
- It is also the highest-risk route-level migration.
- Migrate leaf components first: ‘ChatRow‘, ‘ChatTopBar‘, ‘MessageInput‘, ‘ChatDateHeader‘, and message bubbles.
- Keep pagination and chat lifecycle in Kotlin/graph/interactor until the action boundary is proven.
- Only after leaf parity should the whole chat route become React/Tactile.

Campaign And Discover

Files:

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/campaigns/CampaignScreen.kt

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/campaigns/DiscoverScreen.kt‘

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/components/CampaignCompon

The campaign surface contains:

- current campaign list
- discover campaign list
- search bar
- current campaign card
- discover card

- status chips
- avatar stacks
- gradient action buttons
- TODO actions for join, open chat, and details

Migration implications:

- These are the best first route-level migrations after leaf components.
- They exercise lists, cards, images, search, and buttons without complex lifecycle.
- Existing TODO actions reduce migration risk because not all action flows are final.

Events

Files:

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/events/PrivateEventsScreen.kt‘

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/events/PublicEventsScreen.kt‘

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/events/CreateEventScreen.kt‘

Private events contain:

- static event list
- ‘EventCard‘
- relation label
- action button
- avatar stack
- FAB

Public events contain:

- search
- mutable saved toggle state
- ‘EventRow‘
- thumbnail image
- date and venue text
- FAB

Create event contains:

- event name field
- event type segmented toggle
- date and time fields
- location field
- description field
- character count
- upload cover image button
- create event button

Current limitations:

- Create event is not wired into the graph.
- Many event actions are TODO.

Migration implications:

- Event leaf components are good early candidates.
- Whole event routes should wait until the product event flow is more settled, unless used as a demo-only Tactile proof.

Friends And Profiles

Files:

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/friends/FriendsScreen.kt‘

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/profile/ProfileScreen.kt‘

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/profile/FriendProfileScreen.kt‘

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/profile/GroupProfileScreen.kt‘

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/profile/CreateProfileScreen.kt‘

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/profile/EditProfileScreen.kt‘

The friends list contains:

- friends data fetch
- avatar image
- row click to friend profile

Friend profile contains:

- profile image or placeholder
- top app bar
- age calculation
- bio
- interests FlowRow
- info rows
- conditional message button
- navigation back to chat

Group profile contains:

- group image
- member count
- members list
- current user detection
- click to member friend profile

Own profile contains:

- profile data fetch
- avatar or placeholder
- age and gender display
- editable email, bio, hobbies
- save button TODO

Current limitations:

- Create profile is a stub.
- Edit profile is ‘TODO("Not yet implemented")‘.
- Save profile action is TODO.

Migration implications:

- Profile helpers should migrate as leaves before route-level profile screens.
- Group/friend profile has useful adaptive extra-pane potential for chat.
- Own profile should wait until update behavior is real.

Dev Screen

File:

‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/dev/DevScreen.kt‘

The dev screen contains:

- dev access gating
- native Hermes/C++ test
- FlexBuffer test
- background work test
- analytics test
- crashlytics test

Migration implications:

- This is useful for bridge validation later, but it should not be the first product UI migration.
- It couples UI to native infrastructure status and should be kept stable while the Tactile bridge is under construction.

Product Component Inventory

BestBuds product components live mainly in:

‘/Users/ovd/dev/bestbuds/modules/design/src/commonMain/kotlin/ai/bestbuds/design/components/BestBudsComponent

The main components are:

- ‘Card2‘
- ‘StatusChip‘
- ‘BBText‘
- ‘GradientButton‘
- ‘SearchBar‘
- ‘JoinButton‘
- ‘ActionButton‘
- ‘EventRow‘
- ‘EventCard‘
- ‘TabLayout‘
- ‘BBTextField‘
- ‘EventTypeToggle‘
- ‘SegmentedSwitch‘

Helpers live in:

‘/Users/ovd/dev/bestbuds/modules/design/src/commonMain/kotlin/ai/bestbuds/design/components/BestBudsHelpers.kt‘

The main helpers are:

- ‘LoadingIndicator‘
- ‘ProfilePicturePlaceholder‘
- ‘EditableField‘
- ‘InfoRow‘
- ‘TypewriterText‘
- ‘TypewriterTranslations‘

These are the best first migration layer. They are visible across multiple screens and define product vocabulary. Migrating them first lets the app retain its current Compose route model while the design system becomes real.

Reaktor Desktop Workbench

The Reaktor desktop app starts at:

‘/Users/ovd/dev/bestbuds/targets/reaktorDesktop/src/main/kotlin/Main.kt‘

The workbench is implemented in:

‘/Users/ovd/dev/bestbuds/modules/engine/src/main/kotlin/ai/bestbuds/reaktor/ReaktorWorkbench.kt‘

It has two modes:

- Graph
- Run App

Graph mode renders:

- a Reaktor graph editor
- selected node/graph state

- highlighted node kind
- inspector side panel
- panel tabs: Inspect, Preview, My Graph

Run App mode renders:

- ‘GraphApplication(graph)‘

The inspector panel is in:

‘/Users/ovd/dev/bestbuds/modules/engine/src/main/kotlin/ai/bestbuds/reaktor/ReaktorWorkbenchInspectorPanel.kt‘

It shows:

- node type
- route pattern
- attached node count
- navigation target count
- child graph count
- provider ports
- consumer ports
- attachment rows
- navigation rows

The tree panel is in:

‘/Users/ovd/dev/bestbuds/modules/engine/src/main/kotlin/ai/bestbuds/reaktor/ReaktorWorkbenchTreePanel.kt‘

It shows:

- graph hierarchy
- node rows
- port summaries
- selected graph and node state

The preview panel is in:

‘/Users/ovd/dev/bestbuds/modules/engine/src/main/kotlin/ai/bestbuds/reaktor/ReaktorWorkbenchPreviewPanel.kt‘

It previews ‘ComposeContent‘ nodes and rejects parameterized routes.

The graph document builder is in:

‘/Users/ovd/dev/bestbuds/modules/engine/src/main/kotlin/ai/bestbuds/reaktor/ReaktorGraphDocument.kt‘

It exports:

- visible nodes
- containment edges
- navigation edges
- data edges
- route labels

- graph labels
- provider/consumer port summaries

Migration implications:

- Desktop should become the Tactile developer tool surface.
- It should not be the first product migration target.
- The preview panel should eventually support ‘TactileContent‘.
- The inspector should eventually add a host-tree tab.
- The graph document can later become part of an AI context/tool manifest.

Transition Strategy

The selected transition target is BestBuds mobile.

The selected migration mode is leaf components first.

Phase 1: Define Minimum Host Components

The initial BestBuds host component set should include:

- ‘Text‘
- ‘RawText‘
- ‘Surface‘
- ‘Card‘
- ‘Button‘
- ‘IconButton‘
- ‘Image‘
- ‘Avatar‘
- ‘Chip‘
- ‘TextField‘
- ‘SearchBar‘
- ‘Slider‘
- ‘Progress‘
- ‘Row‘
- ‘Column‘
- ‘Box‘
- ‘LazyList‘
- ‘Scaffold‘
- ‘TopBar‘
- ‘Tabs‘
- ‘FAB‘
- ‘SegmentedSwitch‘
- ‘Divider‘
- ‘ListDetail‘

Do not start with every component imaginable. Start with the host components required by the

current BestBuds UI surface.

Phase 2: Create BestBudsSkin

Create a Tactile product skin that preserves current visual identity:

- black primary from ‘BestBudsTheme‘
- white surfaces
- success green
- current outline and surface variant colors
- product gradients
- spacing scale
- corner scale
- current card border style
- current chip style
- current gradient button style
- current chat bubble style

This phase should not change screen routing. It creates the bridge between the old design vocabulary and the new renderer.

Phase 3: Migrate Leaf Components

Migrate in this order:

1. Text and simple display primitives:
- ‘BBText‘
- ‘StatusChip‘
- ‘ProfilePicturePlaceholder‘

2. Surfaces and buttons:
- ‘Card2‘
- ‘GradientButton‘
- ‘JoinButton‘
- ‘ActionButton‘

3. Inputs:
- ‘SearchBar‘
- ‘BBTextField‘
- ‘SegmentedSwitch‘
- onboarding true/false and single-select controls
- onboarding integer slider

4. List cards:
- ‘EventRow‘
- ‘EventCard‘
- ‘CurrentCampaignCard‘
- ‘DiscoverCard‘

5. Chat components:
- ‘ChatRow‘
- ‘ChatTopBar‘
- ‘MessageInput‘
- ‘ChatDateHeader‘
- sender and receiver message bubbles

6. Profile helpers:
- ‘EditableField‘
- ‘InfoRow‘

This sequence maximizes visual coverage while keeping route-level behavior stable.

Phase 4: Migrate Route-Level Screens

Migrate route screens only after leaf parity is real.

Recommended order:

1. ‘DiscoverScreen‘
2. ‘CampaignScreen‘
3. ‘PrivateEventsScreen‘
4. ‘PublicEventsScreen‘
5. ‘FriendsScreen‘
6. ‘ChatListScreen‘
7. ‘ChatScreen‘
8. ‘FriendProfileScreen‘
9. ‘GroupProfileScreen‘
10. ‘ProfileScreen‘
11. ‘OnboardingScreen‘
12. ‘StartScreen‘

Do not migrate ‘CreateProfileScreen‘ or ‘EditProfileScreen‘ until their product behavior exists.

Phase 5: Adaptive Chat

After chat leaf components are stable, add adaptive chat layout.

Compact behavior remains:

```
/chats -> /chats/{id}
```

Expanded behavior becomes:

```
list pane: /chats
```

```
detail pane: /chats/{id}
extra pane: friend or group profile
```

This requires ‘GraphRouteContent‘ so specific routes can render side by side.

Phase 6: Desktop Tooling

After BestBuds mobile validates the runtime, extend Reaktor Desktop with:

- Tactile host-tree inspector
- reconciler commit viewer
- component catalogue
- skin switcher
- ‘TactileContent‘ preview
- route payload preview harness
- parameterized route preview support

Desktop should become the place to debug and understand the new system.

Testing And Acceptance Criteria

Leaf migration acceptance:

- visual parity with existing Compose components
- stable layout on mobile widths
- semantics preserved where currently present
- no route behavior changes

Interaction acceptance:

- button press invokes handler
- input state changes propagate correctly
- search query updates locally
- segmented switch changes selected index
- slider updates and commits final value
- event save toggle changes state
- message input sends and clears text
- message bubble drag remains native and smooth

Graph acceptance:

- ChatList to ChatScreen works
- ChatScreen back works
- ChatScreen title to FriendProfile works
- ChatScreen title to GroupProfile works
- Friends to FriendProfile works
- FriendProfile to Chat works where chat edge exists

- top-bar overflow still selects Friends/Profile/Dev
- Events tab container still switches Private/Public

Bridge acceptance:

- no per-frame JS/FFI traffic for tactile animation
- handlers are references, not serialized functions
- failed JS bundle or commit has a safe fallback
- existing ComposeContent screens still render during partial migration

Desktop acceptance:

- Graph mode still renders graph editor
- Run App mode still renders BestBuds
- Inspect tab still shows ports and route metadata
- Preview tab still supports ComposeContent
- My Graph tab still shows graph hierarchy
- Tactile tooling additions do not break existing workbench behavior

Key Risks

The biggest product risk is migrating chat too early. Chat mixes route payloads, lifecycle effects, pagination, reverse lists, input, profile navigation, and draggable message bubbles. It should be a major validation target, not the first target.

The biggest design-system risk is failing to centralize BestBuds styling. If ‘BestBudsSkin‘ is not created early, the new system will reproduce the existing split between old ‘reaktor-ui‘, product tokens, and direct Material3 values.

The biggest runtime risk is using JS as the animation driver. That would make the system feel slow and fragile. Native tactile effects must remain native.

The biggest graph risk is trying to replace Reaktor Graph with renderer navigation. Compose Adaptive and RSD layouts should present panes; Reaktor Graph should continue to own semantic navigation.

The biggest tooling risk is waiting too long to inspect host trees. The desktop workbench should gain host-tree inspection once the first real Tactile routes exist.

Assumptions

- BestBuds mobile is the first product transition target.
- Leaf components migrate before whole screens.
- Static JSON SDUI remains out of scope.
- React/Hermes is the composition/runtime model for Compose targets.
- The Tactile host tree is internal reconciler output.
- React Strict DOM is the terminal renderer for React web/native targets.
- Reaktor Graph remains the app runtime.

- Desktop Reaktor becomes tooling support after BestBuds mobile validates the path.

References

Local files:

- ‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/BestBuds.kt‘
- ‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/StartScreen.kt‘
- ‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/onboarding/OnboardingScreen.kt‘
- ‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/chats/ChatScreen.kt‘
- ‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/chats/ChatViews.kt‘
- ‘/Users/ovd/dev/bestbuds/modules/app/src/commonMain/kotlin/ai/bestbuds/app/ui/home/chats/views/MessageView.k
- ‘/Users/ovd/dev/bestbuds/modules/design/src/commonMain/kotlin/ai/bestbuds/design/components/BestBudsCompon
- ‘/Users/ovd/dev/bestbuds/modules/design/src/commonMain/kotlin/ai/bestbuds/design/tokens/BestBudsTokens.kt‘
- ‘/Users/ovd/dev/bestbuds/modules/engine/src/main/kotlin/ai/bestbuds/reaktor/ReaktorWorkbench.kt‘
- ‘/Users/ovd/dev/bestbuds/modules/engine/src/main/kotlin/ai/bestbuds/reaktor/ReaktorWorkbenchPreviewPanel.kt‘
- ‘/Users/ovd/dev/bestbuds/modules/engine/src/main/kotlin/ai/bestbuds/reaktor/ReaktorGraphDocument.kt‘

External references:

- React Strict DOM: https://facebook.github.io/react-strict-dom/
- React reconciler package: https://github.com/facebook/react/tree/main/packages/react-reconciler
- Hermes: https://github.com/facebook/hermes
- Android adaptive layouts: https://developer.android.com/develop/ui/compose/layouts/adaptive
- Cash App Redwood: https://github.com/cashapp/redwood